

Case Study 1 - Cloud-Based Video Conferencing

Analyze whether the problem is primarily caused by the transport protocol (TCP/UDP) or application layer.

The performance issue is caused by both the transport protocol (UDP) & the application layer, but the primary cause is the network conditions affecting UDP communication.

Video conferencing applications use UDP because it provides low latency by transmitting packets without acknowledgements or retransmissions. However, UDP cannot lost packets. The application layer also contributes to the problem because the jitter buffer may not be large enough to compensate for packet delay variations.

2. Justify your answer based on the communication requirements of real-time multimedia applications. Real-time multimedia applications have strict quality of service requirements.

low latency: Voice & video should reach users almost instantly.

low jitter: Packets must arrive at regular intervals.

Minimal packet loss: lost packets reduce audio & video quality.

Continuous communication.

UDP is preferred because:

→ It does not wait for acknowledgements.

→ It avoids retransmission delays.

→ It provides smooth real-time communication.

3. Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.

Protocol level improvements

→ Continue using UDP with RTP for efficient media delivery.

→ Implement FEC to recover lost packets without retransmission.

→ Configure QoS to prioritize Video conferencing traffic.

Application level improvements

→ Implement an adaptive jitter buffer that adjusts automatically according to network conditions.

→ Use efficient codecs such as Opus for audio & H.265/AV for video.

→ Optimize bandwidth allocation during peak business hours.

Case Study 2 - DNS Performance Improvement.

1. Analyze the reasons behind the high DNS resolution time.

The observed DNS lookup time of 800ms is significantly higher than acceptable values.

Possible reasons include:

- Centralized DNS server located far from users.
- Multiple recursive DNS lookups before reaching the authoritative server.
- Poor DNS caching resulting in repeated queries.
- Low TTL configuration causing frequent cache expiration.
- Heavy load on a single DNS server.

2. Propose a suitable DNS architecture using Anycast DNS, DNS pre-fetching, & TTL optimization to reduce lookup time below 50ms.

A distributed DNS architecture should include:

Anycast DNS:

- Deploy multiple DNS servers globally using the same IP address.
- Users are automatically directed to the nearest DNS server.

DNS Pre-fetching

- Browsers resolve frequently accessed domain names before users request them.
- This eliminates waiting time during webpage loading.

Load Balancing

- Distribute DNS requests across multiple authoritative servers.

3. Evaluate the advantages & limitations of your proposed architecture.

Advantages:

- Faster DNS resolution.
- Improved scalability for global users.
- Automatic failover during server failures.
- Reduced latency.
- Lower workload on authoritative DNS servers.

Limitations

- Higher infrastructure & maintenance cost.
- More complex DNS configuration.
- Anycast requires advanced routing support.
- Incorrect TTL values may delay DNS updates.

Case Study 3 - Distributed DNS for Cloud Services.

4. Examine the impact of centralized DNS architecture on application performance.

A centralized DNS architecture directs all DNS requests to a single primary server.

Impact on Performance

- Creates a single point of failure.
- High traffic causes server overload.
- DNS lookup time increases to 800ms.
- Delays application access.
- Increased response time reduces customer satisfaction.

Centralized DNS architecture cannot efficiently support millions of global users because it limits scalability & reliability.

5. Design a distributable DNS solution that minimizes lookup latency while maintaining high availability.

The recommended solution consists of:

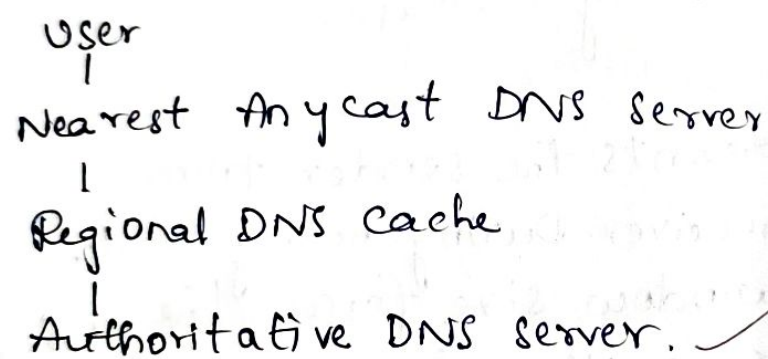
→ Multiple authoritative DNS servers across continents.

→ Anycast routing to connect users to nearest server.

→ Recursive DNS caching at regional locations.

→ Load balancing among DNS servers.

Architecture flow:



6. Analyze how TTL values, Anycast routing, & DNS caching influence system performance & fault tolerance.

TTL

→ Controls cache duration.

→ Reduces repeated DNS queries.

→ Improves response time.

→ Excessively long TTL delays record updates.

Anycast Routing

- Routers users for nearest DNS server.
- Minimizes network latency.

DNS caching

- Stores previously resolved domain names
- Reduces server load.
- Improves Scalability & availability.

Case Study - 4 - TCP latency in stock Trading Platform.

1. Analyze three possible TCP-related factors responsible for the latency increase.

Flow Control.

TCP flow control prevents the sender from overwhelming the receiver. During heavy trading, reduced receiver window size forces the sender to wait before transmitting data.

Nagles Algorithm.

Nagle's algorithm combines multiple small packets before transmission to reduce network overhead.

SYN Queue limitations

When thousands of clients simultaneously establish TCP connections, the server's SYN queue becomes full.

Explain how each factor affects transaction processing in high-frequency systems.

Flow Control:

- Slows data transmission.
- Delays order acknowledgements.
- Reduces transaction throughput

Nagle's Algorithm:

- Buffers small trading packets.
- Delays order execution.

3. Recommend appropriate TCP configuration changes to reduce latency.

Recommended Improvements:

- Disable Nagle's Algorithm using TCP_NODELAY.
- Increase TCP SYN backlog queue size.
- Optimize retransmission timeout (RTO)
- Deploy load balancers to distribute incoming connections.

Expected Benefits:

- Lower acknowledgement latency.
- Faster order processing.
- Improved throughput.
- Better reliability during peak trading hours.
- Higher customer satisfaction